

수요예측 기반 Top-K 후보 구조를 활용한 서울 따릉이 재배치 강화학습

서울 25 개 구 실험에서 A2C 가 가장 안정적인 baseline 대비 개선을 보인 정책경사 비교

작성일: 2026-06-07

Abstract

본 연구는 서울 25 개 구 따릉이 정류소 재배치 문제를 강화학습으로 해결할 수 있는지 검증한다. 재배치 문제는 현재 재고뿐 아니라 앞으로 어느 정류소에서 대여와 반납이 집중될지에 영향을 받는다. 따라서 본 실험에서는 1 시간 수요예측 feature 를 상태 (state)에 추가하고, 매 step 마다 수요예측 기반 Top-K 후보 정류소 12 개를 구성했다.

비교 알고리즘은 REINFORCE with Value Baseline, A2C, PPO 이다. 모든 성능은 고정된 7 개 평가일 평균 reward 와 MostImbalanced baseline 대비 Delta 로 평가했다. 현재 결과에서 A2C 는 평균 Best Delta +16.9, 평균 Final Delta +15.0 로 가장 안정적이었다. 전체 최고 단일 결과는 양천구 / REINFORCE / Best Delta +76.9 이다.

1. 서론

공공 자전거 공유 시스템에서 재배치는 운영 품질을 좌우하는 핵심 문제다. 특정 정류소에 자전거가 부족하면 대여 실패가 발생하고, 특정 정류소가 가득 차면 반납 실패가 발생한다.

초기 실험에서는 단순 RL agent 가 강한 규칙 기반 baseline 을 넘기 어려웠다. 주요 원인은 행동 공간(action space)이 크고, 보상(reward)이 하루 운영 결과로 늦게 반영되며, 현재 상태만으로 미래 수요 집중을 충분히 볼 수 없다는 점이었다. 이에 따라 본 실험은 알고리즘만 바꾸는 방식이 아니라 상태와 행동 구조를 학습 가능한 형태로 재구성하는 방향으로 진행했다.

- 서울 25 개 구를 동일한 방식으로 학습하고 평가한다.
- 1 시간 예측 대여/반납 정보를 상태에 포함한다.
- 전체 정류소 대신 수요예측 기반 Top-K 후보 12 개 중 선택하도록 행동을 구성한다.
- Best checkpoint 와 Final checkpoint 를 분리해 성능 가능성과 안정성을 함께 본다.

2. 관련 연구

공유 자전거 재배치 문제는 vehicle routing, inventory rebalancing, demand forecasting 이 결합된 동적 운영 문제로 연구되어 왔다. Liu et al. (2016)은 multi-source data 를 이용해 정류소별 수요와 재고 목표를 함께 고려했고, TAGCN 계열 연구는 graph 구조와 시간 attention 을 이용해 정류소별 대여/반납 수요를 예측했다.

강화학습 기반 재배치 연구에서도 historical usage, weather, station attributes, demand forecast 를 상태에 포함하는 방향이 자주 사용된다. 본 실험은 이 흐름과 맞닿아 있으며, 특히 agent 가 볼 수 있는 상태에 미래 수요 정보를 넣고 행동 후보를 줄여 학습 신호를 선명하게 만드는 데 초점을 둔다.

3. 용어 정리

용어	의미
MostImbalanced	현재 재고가 목표 재고에서 가장 많이 벗어난 정류소를 우선 방문하는 baseline
Reward	stockout, full, 이동거리 비용을 음수로 합산한 하루 점수. 0 에 가까울수록 좋음
Delta	모델 평가 reward - baseline reward. 양수이면 baseline 보다 좋음
Best checkpoint	학습 중 고정 평가일 평균 reward 가 가장 좋았던 시점
Final checkpoint	학습이 끝난 마지막 시점. Best 와의 차이는 학습 안정성을 보여줌

Top-K action	수요예측 점수 상위 12 개 후보 정류소 중 선택하는 구조
--------------	----------------------------------

4. 문제 정의

서울 25 개 구를 각각 독립된 재배치 실험 단위로 두었다. episode 하나는 하루 운영을 의미하며, 10 분 단위 대여/반납 데이터를 시간 순서대로 replay 하면서 정류소 재고가 변한다. 재배치 agent는 매 decision step마다 다음 방문 정류소를 선택한다.

항목	정의
State	현재 재고, capacity, target 편차, 1 시간 예측 수요, 트럭 상태, 시간 정보
Action	현재 step의 Top-K 후보 중 다음 방문 정류소 선택
Reward	$-1.0 * \text{stockout} - 0.8 * \text{full} - 0.008 * \text{travel_km} - 0.002 * \text{travel_step}$
Evaluation	고정 7 개 날짜 평균 reward와 baseline 대비 Delta

```

pred_net_1h = pred_returns_1h - pred_rentals_1h
projected_bikes = current_bikes + pred_net_1h
projected_deviation = (projected_bikes - target_bikes) / capacity

Delta = model_eval_reward - MostImbalanced_eval_reward

```

5. 방법론

전처리된 서울 전체 데이터는 정류소 테이블과 10 분 단위 대여/반납 테이블로 구성된다. 구별 수요예측 파일은 각 시각과 정류소의 1 시간 예측 대여량, 반납량, 순수요를 담는다.

데이터	현재 보고서 기준
구 수	25.0
분석 대상 정류소 수	3313.0
active 정류소 수	2808.0
10 분 대여/반납 row 수	40,565,021
구별 forecast parquet	25 개

Top-K 구조는 agent가 정류소 전체를 무작위로 탐색하지 않도록 돕는다. 후보는 예측 불균형, 이동거리, 권역 penalty를 함께 고려해 만들어진다. policy network의 행동 index는 고정 정류소 ID가 아니라 현재 step의 후보 rank를 의미한다.

```

candidate_score =
    forecast_imbalance
    - candidate_travel_coef * travel_distance
    - zone_penalty

```

6. 알고리즘

6.1 REINFORCE with Value Baseline

REINFORCE는 episode 종료 후 reward-to-go를 계산하여 policy를 업데이트하는 Monte Carlo policy gradient 알고리즘이다. Value network는 baseline으로 사용해 advantage의 분산을 줄인다.

```

returns = discounted_reward_to_go(rewards, gamma)
advantages = returns - value_net(states)

```

```
policy_loss = -(log_probs * advantages.detach()).mean()
value_loss = mse_loss(value_net(states), returns)
```

6.2 A2C

A2C 는 actor(policy)와 critic(value)을 함께 학습한다. TD target 을 사용하므로 REINFORCE 보다 더 자주 업데이트할 수 있고, 이번 실험에서는 가장 안정적인 평균 성능을 보였다.

```
target = reward + gamma * (1 - done) * value(next_state)
advantage = target - value(state)

actor_loss = -(log_prob(action) * advantage.detach()).mean()
critic_loss = mse_loss(value(state), target)
```

6.3 PPO

PPO 는 policy 가 한 번에 너무 크게 변하지 않도록 clipped objective 를 사용한다. 본 실험에서는 action mask 를 지원하는 MaskablePPO 를 사용해 Top-K 후보 밖의 action 은 선택되지 않도록 했다.

```
r_t(theta) = pi_theta(a_t | s_t) / pi_old(a_t | s_t)
L_clip = min(
    r_t(theta) * A_t,
    clip(r_t(theta), 1 - epsilon, 1 + epsilon) * A_t
)
```

7. 실험 설정

항목	값
범위	서울 25 개 구
학습 데이터	구별 train pool 200 일
평가 날짜	2025-03-25, 2025-04-18, 2025-05-17, 2025-07-01, 2025-07-06, 2025-07-09, 2025-08-21
평가 지표	7 개 평가일 평균 reward
baseline	MostImbalanced
후보 action 수	Top-K 12
수요예측 horizon	1 시간
BC 사용 여부	현재 결과는 no-BC
rollback 사용 여부	현재 interactive full run 은 rollback 없음

알고리즘	주요 하이퍼파라미터
REINFORCE	gamma=0.99, hidden=256, lr_policy=3e-4, lr_value=1e-3, normalize_advantages=True
A2C	gamma=0.99, hidden=256, lr_policy=1e-4, lr_value=3e-4, batch_size=32, memory_size=200
PPO	gamma=0.99, gae_lambda=0.95, lr=1e-4, clip_range=0.1, ent_coef=0.003, target_kl=0.03

8. 실험 결과

8.1 알고리즘별 전체 요약

모든 결과는 고정 7 개 평가일에서 MostImbalanced baseline 대비 Delta 로 비교했다. Best checkpoint 는 성능 가능성, Final checkpoint 는 학습 안정성을 의미한다.

Algorithm	구 수	Best 승리 구	Final 승리 구	Mean Best Δ	Median Best Δ	Mean Final Δ
A2C	25.0	17.0	17.0	16.9	13.0	15.0
REINFORCE	25.0	13.0	9.0	-0.8	3.9	-27.4
PPO	25.0	13.0	10.0	-3.5	3.9	-35.7

Figure 1 은 기존 막대 분포 대신 구별 scorecard 로 구성했다. 왼쪽 세 열은 알고리즘별 Best Delta 이고, 오른쪽은 정류소 수, 전체 수요량, forecast coverage 이다. 붉은 셀이 몰린 구는 baseline 을 넘지 못한 구이며, 오른쪽 지표를 함께 보면 단순 알고리즘 문제 인지, 수요 규모가 큰 구의 reward scale 문제인지, 예측 데이터 coverage 가 낮은 문제인지 비교할 수 있다.

구별 Best Delta와 데이터 특성 Scorecard

붉은 셀은 baseline 하회, 푸른 셀은 baseline 초과입니다. 오른쪽 지표는 하라 구간의 규모/복잡도/예측 커버리지를 비교하기 위한 보조 정보입니다.

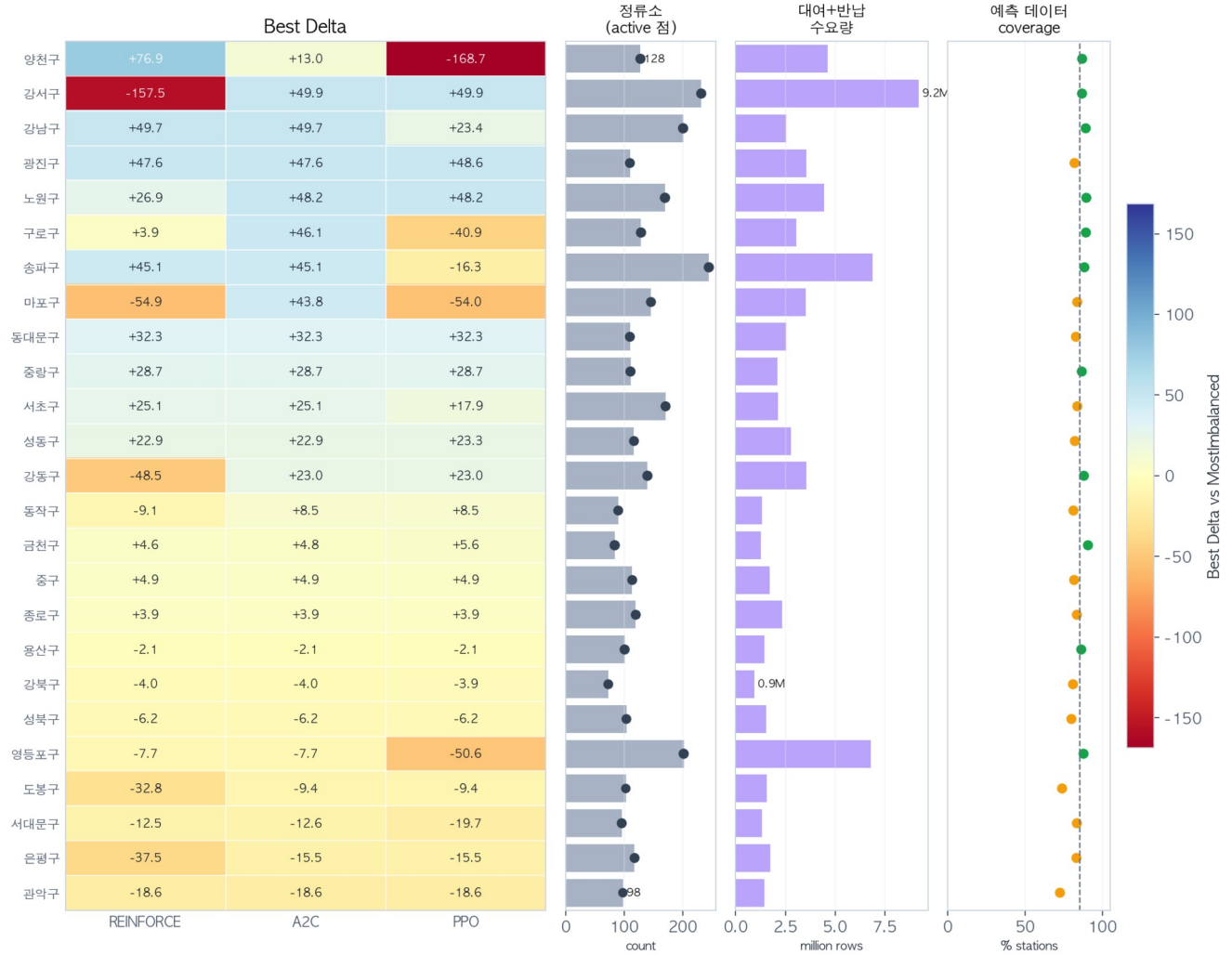


Figure 1. 구별 Best Delta와 데이터 특성 Scorecard

8.2 학습곡선

아래 그림은 train reward 가 아니라, 학습 중 주기적으로 고정 평가일을 다시 실행한 주기적 평가 return 이다. 실선은 25 개 구 평균 Delta, 점선은 중앙값, 음영은 IQR 이다.

25개 구 주기적 평가 return 학습곡선

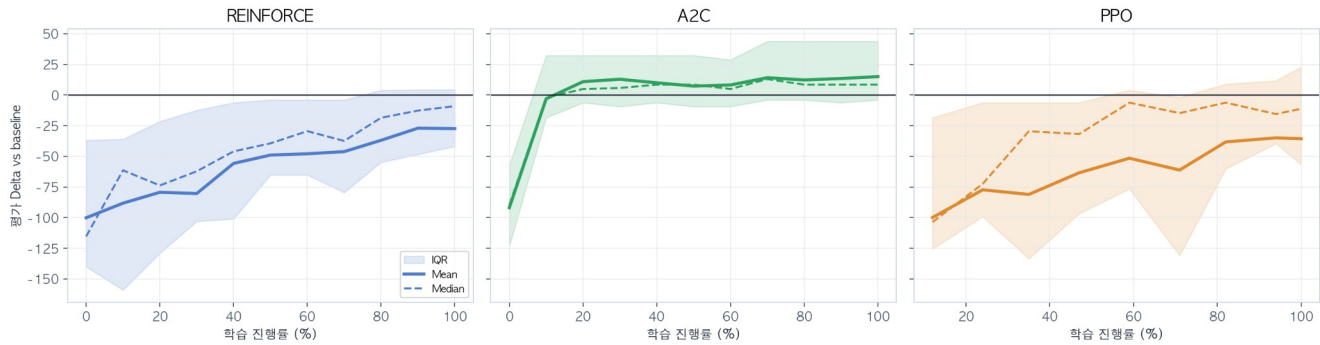


Figure 2. REINFORCE/A2C/PPO 주기적 평가 return 학습곡선

8.3 Best 3 / Worst 3 구

Algorithm	구분	구	Baseline	Best	Final	Best Δ	Final Δ	Best step
A2C	Best	강서구	-3549.0	-3499.1	-3499.1	49.9	49.9	50.0
A2C	Best	강남구	-531.7	-482.0	-482.0	49.7	49.7	50.0
A2C	Best	노원구	-800.3	-752.1	-752.1	48.2	48.2	50.0
A2C	Worst	관악구	-143.5	-162.1	-162.1	-18.6	-18.6	50.0
A2C	Worst	은평구	-241.8	-257.3	-257.3	-15.5	-15.5	50.0
A2C	Worst	서대문구	-191.4	-204.0	-204.2	-12.6	-12.7	200.0
PPO	Best	강서구	-3549.0	-3499.1	-3668.7	49.9	-119.6	40000.0
PPO	Best	광진구	-899.5	-850.9	-850.9	48.6	48.6	170000.0
PPO	Best	노원구	-800.3	-752.1	-752.1	48.2	48.2	20000.0
PPO	Worst	양천구	-1511.2	-1679.9	-1731.2	-168.7	-219.9	160000.0
PPO	Worst	마포구	-760.2	-814.2	-894.6	-54.0	-134.4	140000.0
PPO	Worst	영등포구	-2440.1	-2490.7	-2602.1	-50.6	-162.0	40000.0
REINFORCE	Best	양천구	-1511.2	-1434.3	-1652.9	76.9	-141.7	50.0
REINFORCE	Best	강남구	-531.7	-482.0	-482.0	49.7	49.7	450.0
REINFORCE	Best	광진구	-899.5	-851.9	-851.9	47.6	47.6	200.0
REINFORCE	Worst	강서구	-3549.0	-3706.5	-3706.5	-157.5	-157.5	500.0
REINFORCE	Worst	마포구	-760.2	-815.2	-826.3	-54.9	-66.1	300.0
REINFORCE	Worst	강동구	-662.9	-711.4	-711.4	-48.5	-48.5	350.0

아래 그림은 알고리즘별 Best/Worst 3 구를 분리한 것이다. 각 박스는 하나의 구이며, 초록 배경은 Best 3, 붉은 배경은 Worst 3 을 의미한다. 검은 점은 해당 구에서 가장 좋았던 평가 시점을 나타낸다.

A2C: Best/Worst 3 구별 학습곡선

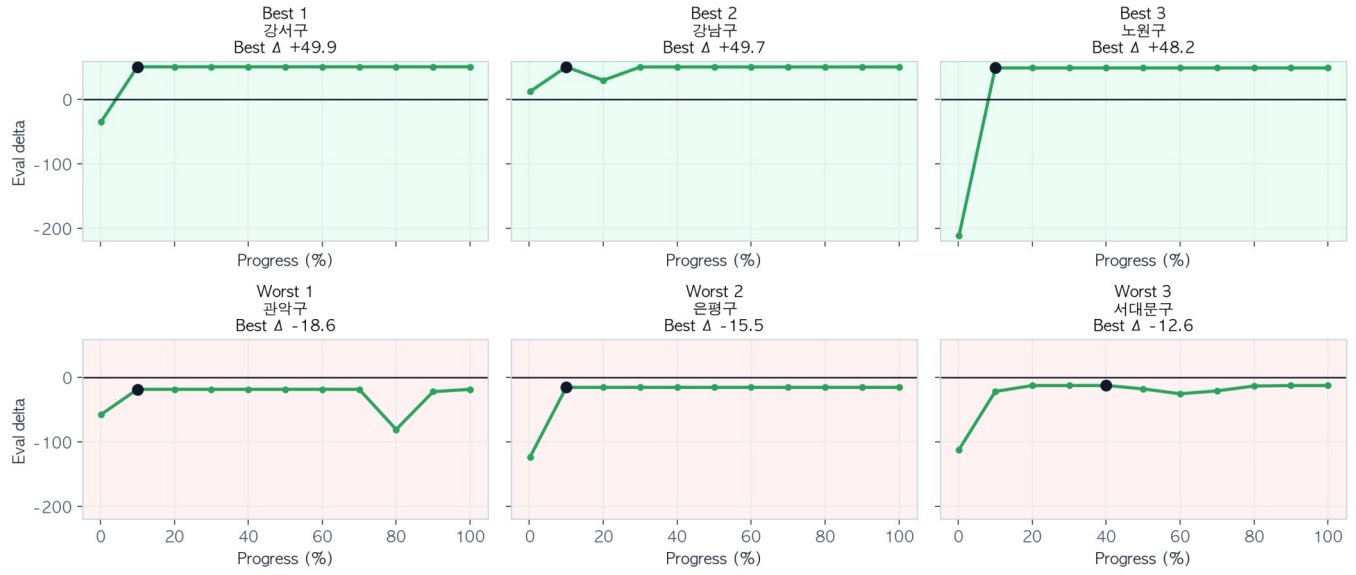


Figure 3a. A2C Best/Worst 3 구별 학습곡선

REINFORCE: Best/Worst 3 구별 학습곡선

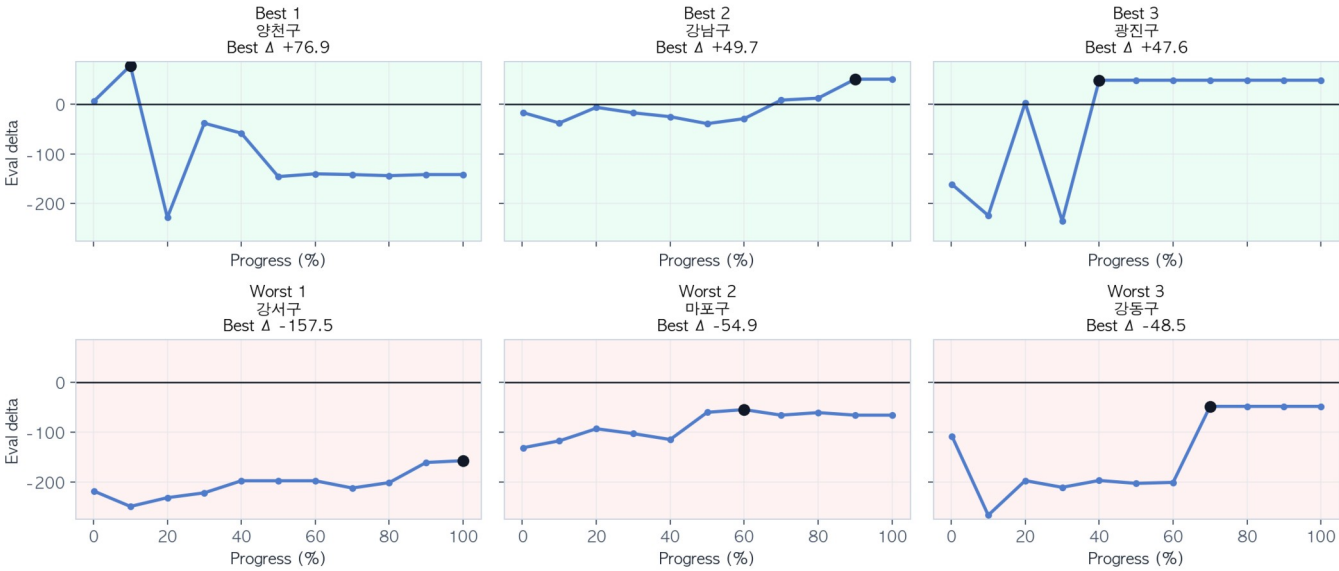


Figure 3b. REINFORCE Best/Worst 3 구별 학습곡선

PPO: Best/Worst 3 구별 학습곡선

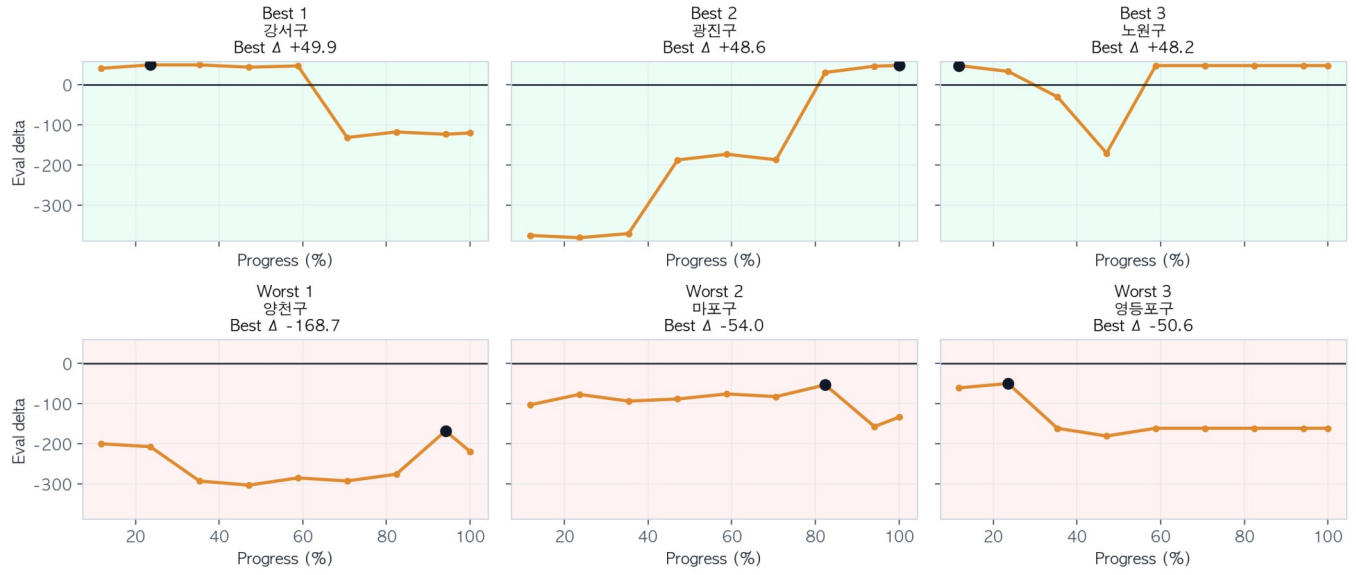


Figure 3c. PPO Best/Worst 3 구별 학습곡선

8.4 서울 지도와 원인 분석

서울 25개 구별 최고 알고리즘과 Best delta

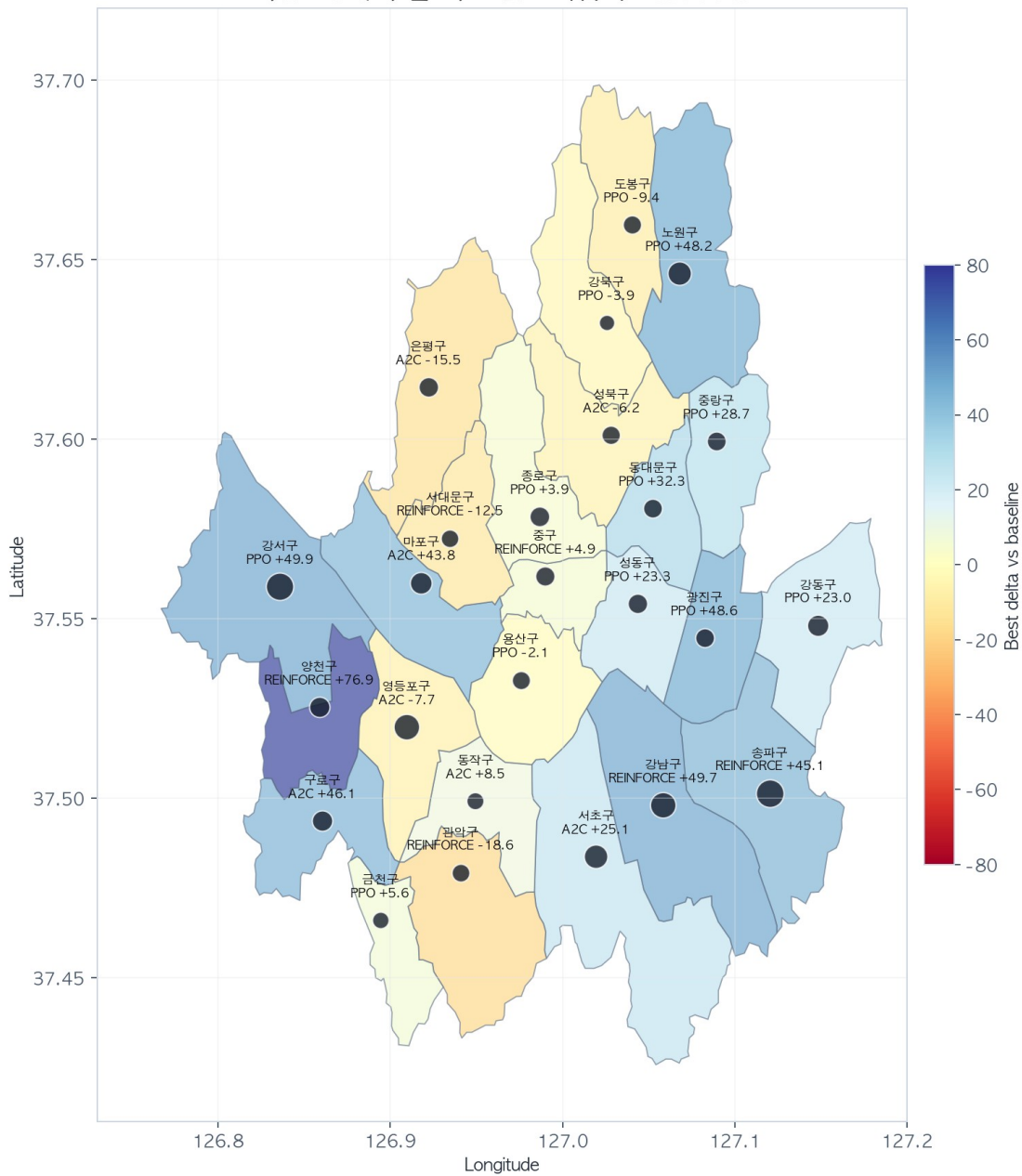


Figure 4. 서울 25개 구별 최고 알고리즘과 Best Delta

Best/Worst 결과 해석용 scatter

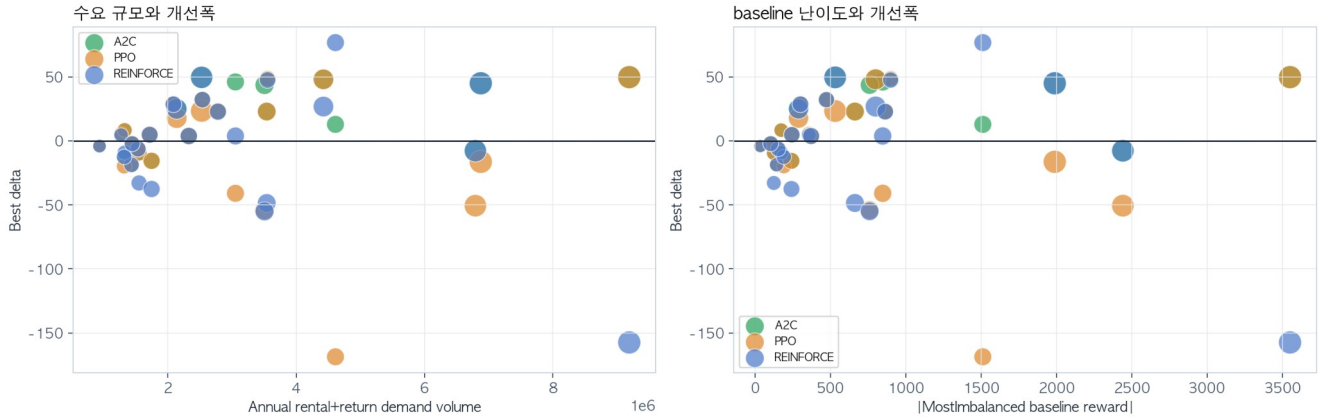


Figure 5. 수요 규모와 baseline 난이도 대비 Best Delta

9. 논의

구별 성능 차이는 수요의 시공간 집중도, baseline 난이도, reward scale 차이로 해석할 수 있다. 특정 시간과 지역에 수요가 강하게 몰리면 Top-K 후보가 실제 문제 정류소를 잘 잡을 때 개선폭이 커진다. 반대로 MostImbalanced가 이미 잘 작동하는 구에서는 RL이 추가로 개선할 여지가 작다.

A2C는 평균 Best Delta와 Final Delta가 모두 가장 안정적이다. TD 기반 advantage를 사용하기 때문에 episode 전체 reward를 기다리는 REINFORCE보다 업데이트 신호가 빠르고, PPO보다 현재 Top-K rank 행동 구조에 덜 민감하게 작동한 것으로 해석된다.

REINFORCE는 일부 구에서 큰 개선을 만들지만 Final 안정성이 낮다. Monte Carlo return을 사용하기 때문에 reward 분산이 크고, 구별 수요 패턴에 따라 학습곡선의 흔들림이 커질 수 있다.

PPO는 clipping과 target KL을 사용하므로 일반적으로 안정적이라고 알려져 있지만, 이번 구조에서는 구별 편차가 컸다. Top-K rank 행동은 매 step마다 행동 index의 의미가 바뀌므로, PPO가 기대하는 완만한 policy update가 항상 좋은 방향으로 누적되지 않을 수 있다.

10. 결론

현재 25개 구 결과에서 REINFORCE, A2C, PPO를 비교하면 A2C가 가장 안정적인 선택이다. REINFORCE는 개선 가능성은 크지만 구별 편차가 있고, PPO는 Best checkpoint 기준 가능성은 있으나 Final 안정성이 약하다.

이 결과의 함의는 단순히 어떤 알고리즘이 가장 좋은가에서 끝나지 않는다. 향후 연구에서는 구별 독립 학습을 넘어 구간 이동, 트럭 배치 수, depot 위치, 실시간 재고 스냅샷을 함께 반영해야 한다. 또한 Top-K 후보가 실패한 구에서는 후보 생성 점수와 수요 예측 coverage를 함께 진단해야 한다.

가장 중요한 결론은 서울 따릉이 재배치 문제에서는 알고리즘 선택만큼 상태와 행동 구조 설계가 중요하다는 점이다. 1시간 수요 예측과 Top-K 후보 행동 구조는 RL이 baseline을 넘어설 수 있는 조건을 만들었고, 그 효과는 구별 수요 패턴과 baseline 난이도에 따라 다르게 나타났다.

References

1. Liu, J. et al. (2016). Rebalancing Bike Sharing Systems: A Multi-source Data Smart Optimization. KDD.
2. Chai, D. et al. (2021). TAGCN: Station-level demand prediction for bike-sharing system via a temporal attention graph convolution network. Information Sciences.
3. Pan, L. et al. (2024). A Reinforcement Learning Approach for Dynamic Rebalancing in Bike-Sharing System. arXiv:2402.03589.

4. Betkier, I., & Dawid, W. (2025). Intelligent Rebalancing: Reinforcement Learning Agent for Optimal Bike-Sharing Distribution Powered by Historical Usage Data. SSRN.
5. Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
6. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning.
7. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction.
8. Seoul administrative boundary GeoJSON. <https://github.com/southkorea/seoul-maps>

Appendix A. 전체 구별 결과

Algorithm	구	Baseline	Best	Final	Best Δ	Final Δ	Best step
A2C	강서구	-3549.0	-3499.1	-3499.1	49.9	49.9	50.0
A2C	강남구	-531.7	-482.0	-482.0	49.7	49.7	50.0
A2C	노원구	-800.3	-752.1	-752.1	48.2	48.2	50.0
A2C	광진구	-899.5	-851.9	-851.9	47.6	47.6	100.0
A2C	구로구	-847.1	-800.9	-800.9	46.1	46.1	50.0
A2C	송파구	-1988.4	-1943.3	-1943.3	45.1	45.1	50.0
A2C	마포구	-760.2	-716.5	-716.5	43.8	43.8	50.0
A2C	동대문구	-473.8	-441.6	-441.6	32.3	32.3	50.0
A2C	종로구	-301.3	-272.6	-272.6	28.7	28.7	100.0
A2C	서초구	-288.8	-263.8	-285.4	25.1	3.5	50.0
A2C	강동구	-662.9	-639.9	-639.9	23.0	23.0	50.0
A2C	성동구	-863.7	-840.8	-840.8	22.9	22.9	50.0
A2C	양천구	-1511.2	-1498.2	-1498.2	13.0	13.0	200.0
A2C	동작구	-174.0	-165.5	-165.5	8.5	8.5	150.0
A2C	중구	-244.3	-239.4	-239.4	4.9	4.9	100.0
A2C	금천구	-357.1	-352.3	-352.5	4.8	4.6	100.0
A2C	종로구	-371.9	-368.0	-368.0	3.9	3.9	50.0
A2C	용산구	-104.4	-106.5	-106.5	-2.1	-2.1	50.0
A2C	강북구	-35.2	-39.2	-39.2	-4.0	-4.0	50.0
A2C	성북구	-154.3	-160.5	-160.5	-6.2	-6.2	50.0
A2C	영등포구	-2440.1	-2447.8	-2471.7	-7.7	-31.6	100.0
A2C	도봉구	-124.9	-134.3	-134.3	-9.4	-9.4	50.0
A2C	서대문구	-191.4	-204.0	-204.2	-12.6	-12.7	200.0
A2C	은평구	-241.8	-257.3	-257.3	-15.5	-15.5	50.0
A2C	관악구	-143.5	-162.1	-162.1	-18.6	-18.6	50.0
PPO	강서구	-3549.0	-3499.1	-3668.7	49.9	-119.6	40000.0
PPO	광진구	-899.5	-850.9	-850.9	48.6	48.6	170000.0
PPO	노원구	-800.3	-752.1	-752.1	48.2	48.2	20000.0
PPO	동대문구	-473.8	-441.6	-441.6	32.3	32.3	80000.0
PPO	종로구	-301.3	-272.6	-272.6	28.7	28.7	100000.0
PPO	강남구	-531.7	-508.3	-508.3	23.4	23.4	170000.0
PPO	성동구	-863.7	-840.4	-840.8	23.3	22.9	140000.0
PPO	강동구	-662.9	-639.9	-639.9	23.0	23.0	160000.0
PPO	서초구	-288.8	-270.9	-270.9	17.9	17.9	170000.0
PPO	동작구	-174.0	-165.5	-172.8	8.5	1.2	40000.0
PPO	금천구	-357.1	-351.4	-387.9	5.6	-30.9	140000.0
PPO	중구	-244.3	-239.4	-239.4	4.9	4.9	170000.0
PPO	종로구	-371.9	-368.0	-395.1	3.9	-23.2	120000.0
PPO	용산구	-104.4	-106.5	-106.5	-2.1	-2.1	60000.0
PPO	강북구	-35.2	-39.2	-43.4	-3.9	-8.2	20000.0
PPO	성북구	-154.3	-160.5	-211.0	-6.2	-56.7	20000.0
PPO	도봉구	-124.9	-134.3	-136.2	-9.4	-11.2	60000.0
PPO	은평구	-241.8	-257.3	-257.3	-15.5	-15.5	160000.0
PPO	송파구	-1988.4	-2004.7	-2201.8	-16.3	-213.3	20000.0
PPO	관악구	-143.5	-162.1	-162.1	-18.6	-18.6	20000.0
PPO	서대문구	-191.4	-211.1	-224.0	-19.7	-32.6	80000.0
PPO	구로구	-847.1	-888.0	-941.1	-40.9	-94.0	120000.0
PPO	영등포구	-2440.1	-2490.7	-2602.1	-50.6	-162.0	40000.0

PPO	마포구	-760.2	-814.2	-894.6	-54.0	-134.4	140000.0
PPO	양천구	-1511.2	-1679.9	-1731.2	-168.7	-219.9	160000.0
REINFORCE	양천구	-1511.2	-1434.3	-1652.9	76.9	-141.7	50.0
REINFORCE	강남구	-531.7	-482.0	-482.0	49.7	49.7	450.0
REINFORCE	광진구	-899.5	-851.9	-851.9	47.6	47.6	200.0
REINFORCE	송파구	-1988.4	-1943.3	-1943.3	45.1	45.1	50.0
REINFORCE	동대문구	-473.8	-441.6	-441.6	32.3	32.3	100.0
REINFORCE	종로구	-301.3	-272.6	-297.0	28.7	4.3	50.0
REINFORCE	노원구	-800.3	-773.4	-957.9	26.9	-157.6	150.0
REINFORCE	서초구	-288.8	-263.8	-263.8	25.1	25.1	250.0
REINFORCE	성동구	-863.7	-840.8	-905.6	22.9	-41.9	50.0
REINFORCE	중구	-244.3	-239.4	-239.4	4.9	4.9	500.0
REINFORCE	금천구	-357.1	-352.5	-391.6	4.6	-34.5	450.0
REINFORCE	구로구	-847.1	-843.2	-843.2	3.9	3.9	400.0
REINFORCE	종로구	-371.9	-368.0	-368.0	3.9	3.9	200.0
REINFORCE	용산구	-104.4	-106.5	-106.5	-2.1	-2.1	400.0
REINFORCE	강북구	-35.2	-39.2	-39.2	-4.0	-4.0	100.0
REINFORCE	성북구	-154.3	-160.5	-160.5	-6.2	-6.2	150.0
REINFORCE	영등포구	-2440.1	-2447.8	-2571.6	-7.7	-131.5	150.0
REINFORCE	동작구	-174.0	-183.0	-183.0	-9.1	-9.1	500.0
REINFORCE	서대문구	-191.4	-203.9	-204.2	-12.5	-12.7	150.0
REINFORCE	관악구	-143.5	-162.1	-162.1	-18.6	-18.6	250.0
REINFORCE	도봉구	-124.9	-157.8	-157.8	-32.8	-32.8	500.0
REINFORCE	은평구	-241.8	-279.3	-279.3	-37.5	-37.5	450.0
REINFORCE	강동구	-662.9	-711.4	-711.4	-48.5	-48.5	350.0
REINFORCE	마포구	-760.2	-815.2	-826.3	-54.9	-66.1	300.0
REINFORCE	강서구	-3549.0	-3706.5	-3706.5	-157.5	-157.5	500.0

Appendix B. 재현용 주요 실행 설정

```
PYTHONUNBUFFERED=1 PYTHONPATH=. .venv/bin/python -m src.agents.ours.run_interactive
```

```
processed_dir = data/processed_seoul_all
forecast_dir = data/forecast_by_gu
capacity_path = data/processed/station_capacity.csv
future_mode = forecast_projected_travel
candidate_top_k = 12
candidate_mode = forecast_imbalance
candidate_travel_coef = 0.20
candidate_zone_mode = static3
bc_epochs = 0
```